William Stallings
Computer Organization and Architecture
6th Edition

Chapter 3
Computer Function and Buses

The Von Neumann Model/Architecture

- Also called stored program computer (instructions in memory).
- Two key properties:
  - Stored program
    - Instructions stored in a linear memory array
    - Memory is unified between instructions and data
      - The interpretation of a stored value depends on the control signals
        - When is a value interpreted as an instruction?
  - Sequential instruction processing
    - One instruction processed (fetched, executed, and completed) at a time

A computing System (Reminder)

- What is a computer?
  - in terms of what?
    - Functional
    - Structural

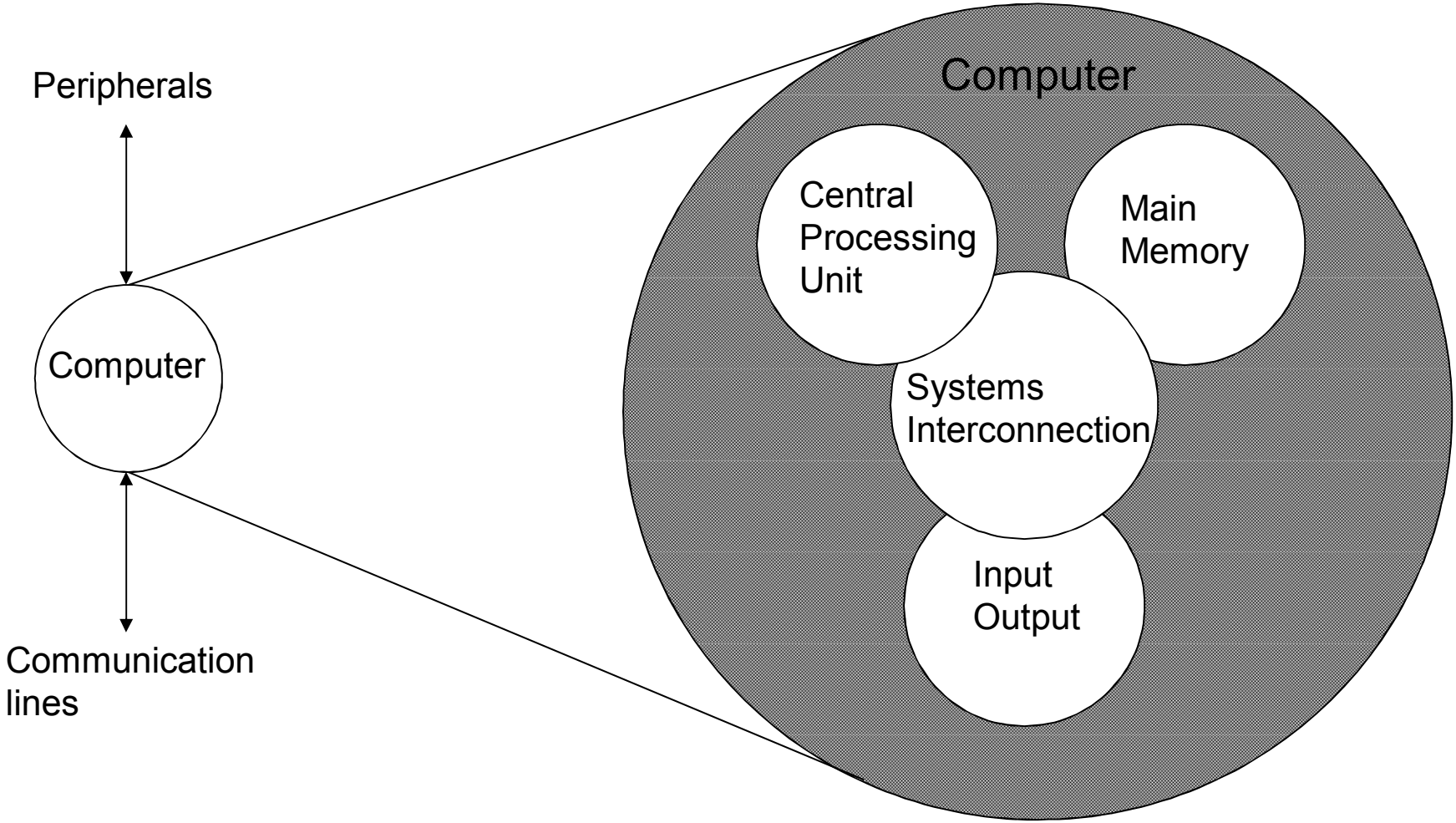
Functional definition

- Data processing
- Data storage
- Data movement
- Control

Structural definition

- Central processing unit
- Main memory
- Input/Output
- System interconnection

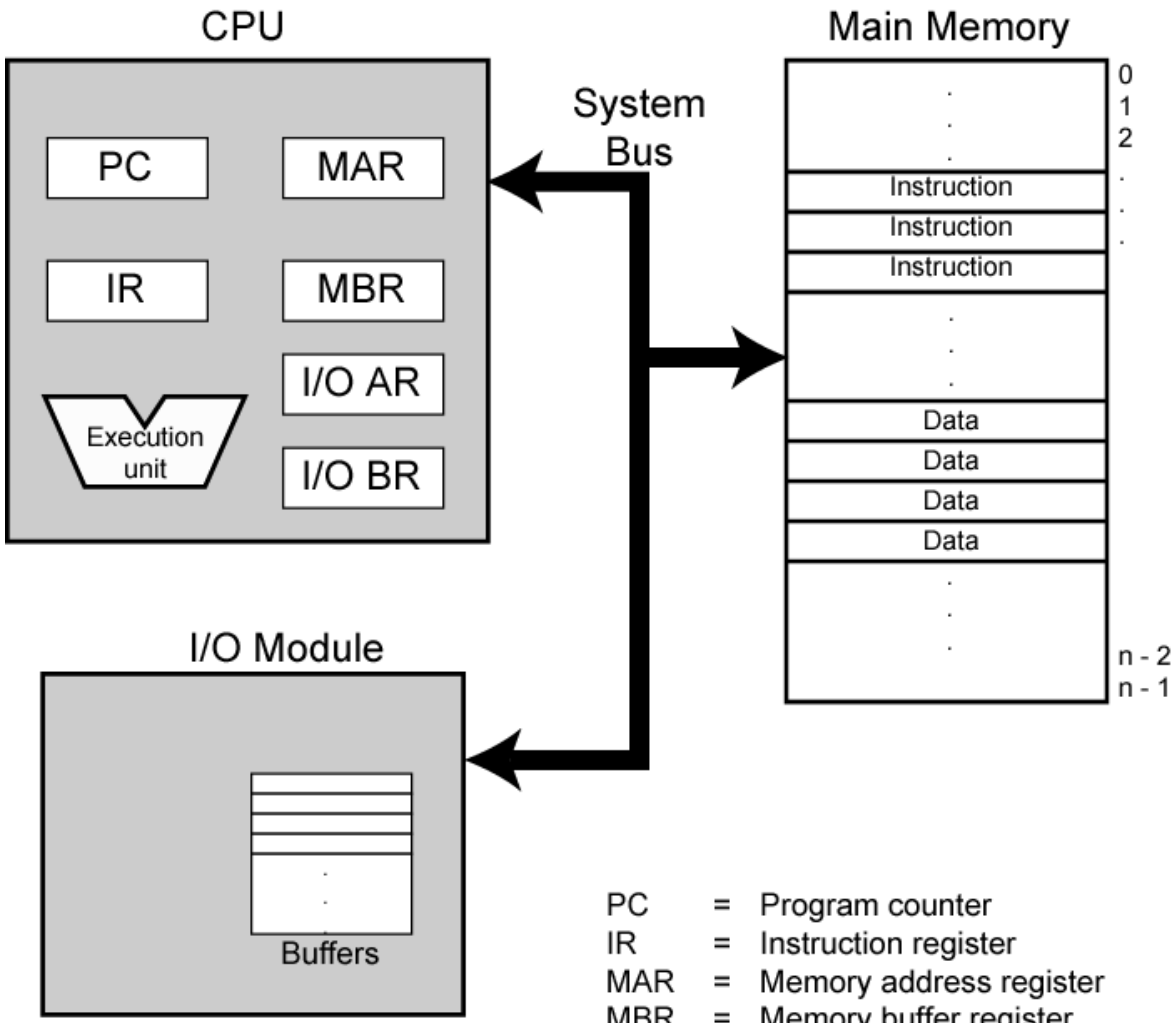
Structure - Top Level



The Stored Program Computer

- 1943: ENIAC
  - Presper Eckert and John Mauchly -- first general electronic computer. **(or was it John V. Atanasoff in 1939?)**
  - Hard-wired program -- settings of dials and switches.
- 1944: Beginnings of EDVAC
  - among other improvements, includes program stored in memory
- 1945: John von Neumann
  - wrote a report on the stored program concept, known as the *First Draft of a Report on EDVAC*
- The basic structure proposed in the draft became known as the "von Neumann machine" (or model).
  - a *memory*, containing instructions and data
  - a *processing unit*, for performing arithmetic and logical operations
  - a *control unit*, for interpreting instructions

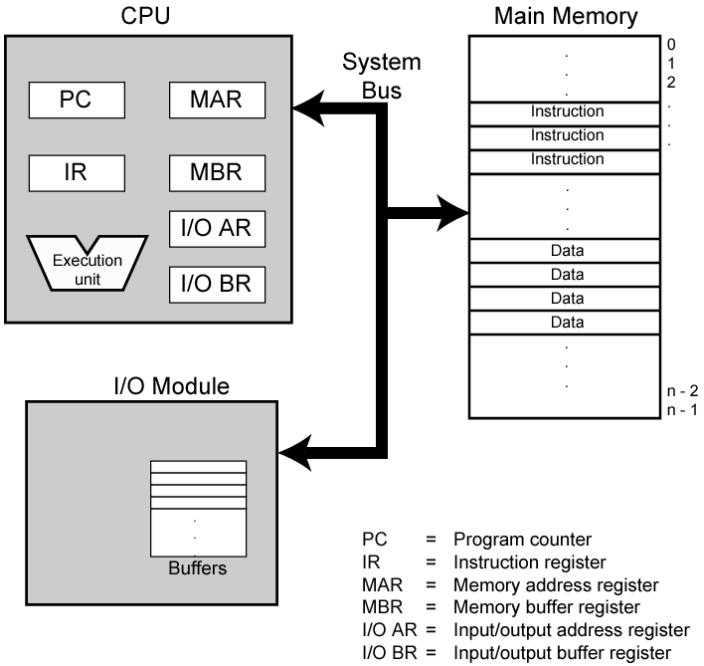
Von Neumann Model



PC = Program counter
IR = Instruction register
MAR = Memory address register
MBR = Memory buffer register
I/O AR = Input/output address register
I/O BR = Input/output buffer register

Interface to Memory

- How does processing unit get data to/from memory?
- MAR: Memory Address Register
- MBR (MDR): Memory Buffer (Data) Register



To LOAD a memory location (A):

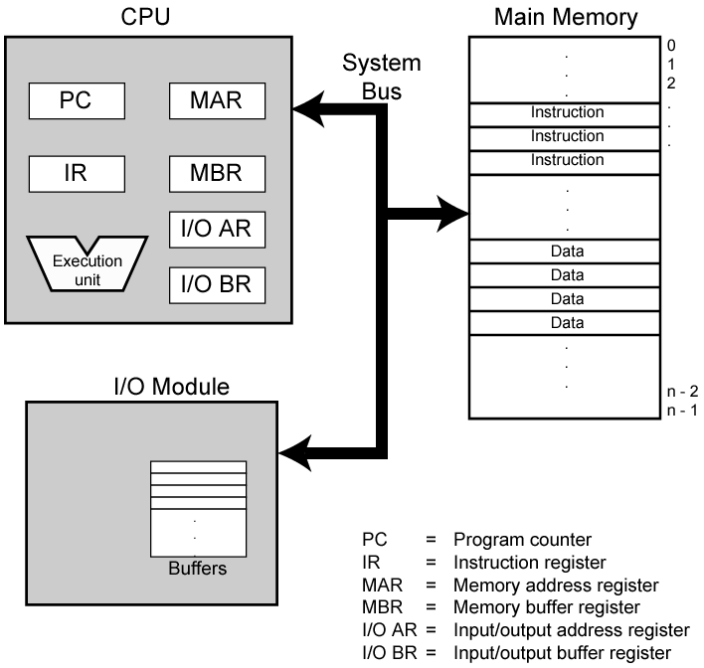
1. Write the address (A) into the MAR.
2. Send a "read" signal to the memory.
3. Read the data from MBR.

To STORE a value (X) to a location (A):

1. Write the data (X) to the MBR.
2. Write the address (A) into the MAR.
3. Send a "write" signal to the memory.

Processing Unit

- Functional Units (Execution unit)
  - ALU = Arithmetic and Logic Unit
  - could have many functional units. some of them special-purpose (multiply, square root, …)
- Registers
  - Small, temporary storage
  - Operands and results of functional units
- Word Size
  - number of bits normally processed by ALU in one instruction
  - also width of registers



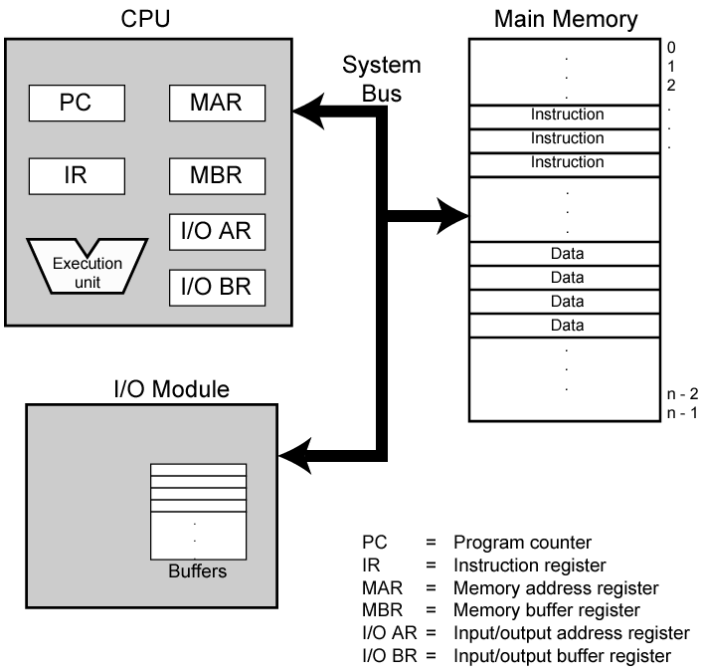
Input and Output

- Devices for getting data into and out of computer memory

- Each device has its own interface, usually a set of registers (I/OAR and I/OBR)

- Some devices provide both input and output
  - disk, network

- Program that controls access to a device is usually called a *driver*.



Control Unit

•Orchestrates execution of the program

•Instruction Register (IR) contains the *current instruction*.

•Program Counter (PC) contains the *address* of the next instruction to be executed.

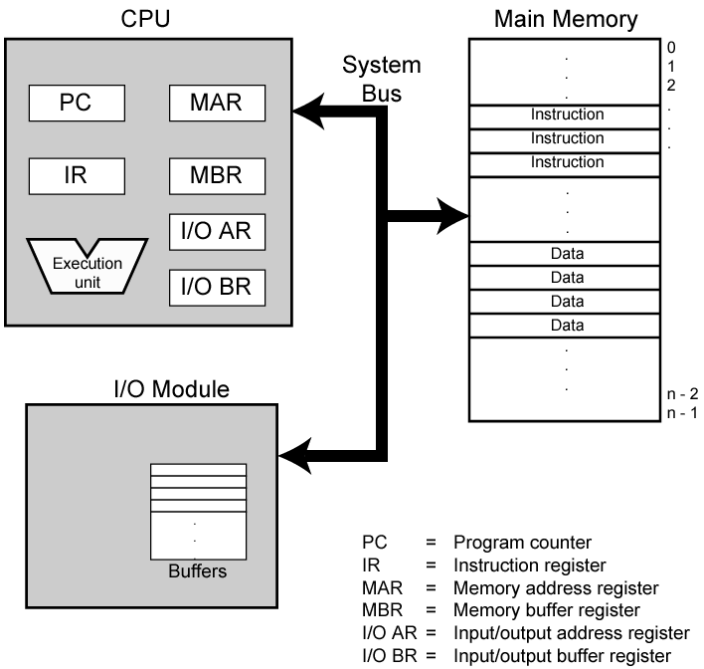
•Control unit:

– reads an instruction from memory

- the instruction's address is in the PC

– interprets the instruction, generating signals that tell the other components what to do

- an instruction may take many *machine cycles* to complete

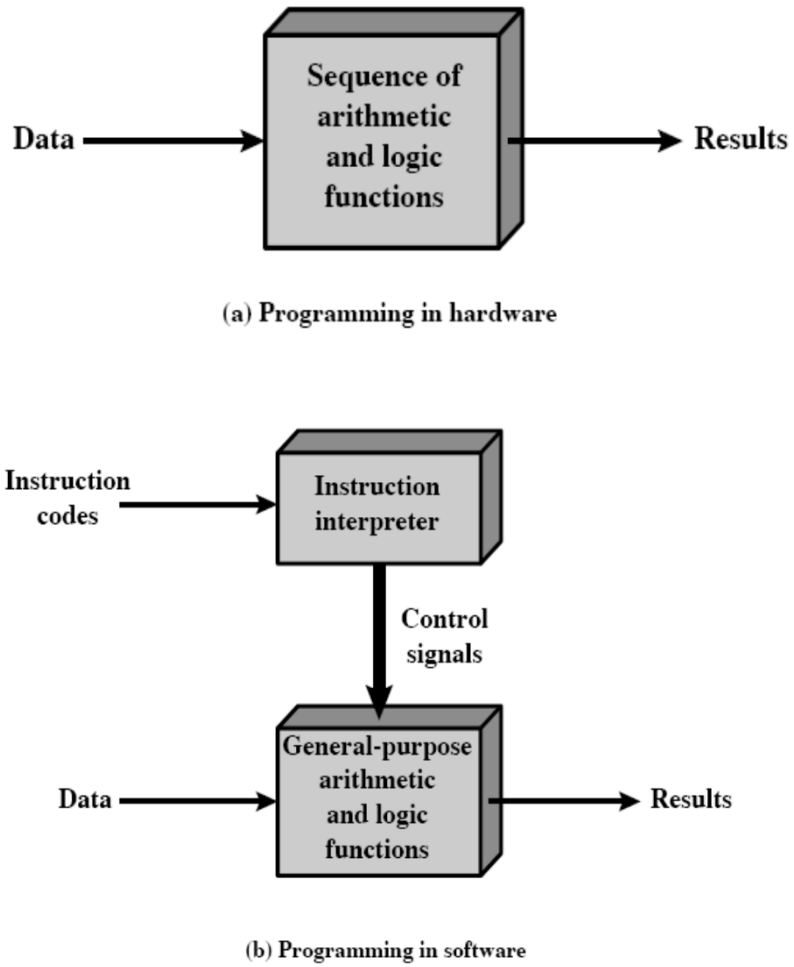


Function of Control Unit

- For each operation unique code is provided (ADD, MOVE), that generates coorect control signals
- A hardware segment(**control unit**) accepts the code and issues the control signals
- Typical outputs from control unit
  - **Drive signals:** DrPC, DrALU, DrREG, DrMEM
  - **Load signals:** LdPC, LdMAR, LdIR
  - **Write Memory signal:** WrMEM
  - **Write Registers signal:** WrREG
  - **ALU function selector:** func
  - **Register selector:** regno
- We have a computer!

Program Concept

- Hardwired systems are inflexible
- General purpose hardware can do different tasks, given correct control signals
- Instead of re-wiring, supply a new set of control signals

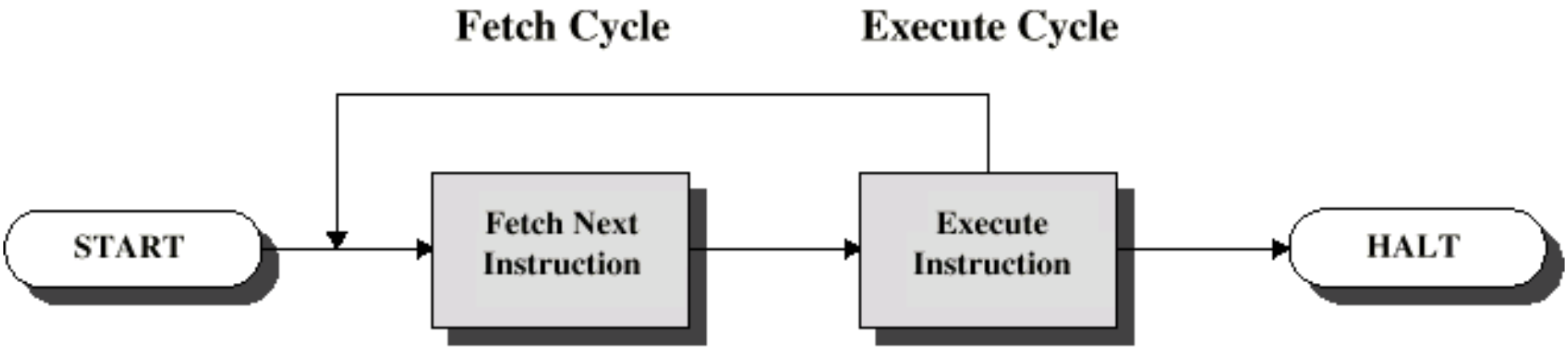


What is a program?

- A sequence of steps
- For each step, an arithmetic or logical operation is done
- For each operation, a different set of control signals is needed

CPU Instruction Cycle (Basic)

- Two steps:
  - Fetch
  - Execute



Fetch Cycle

- Program Counter (PC) holds address of next instruction to fetch
- Processor fetches instruction from memory location pointed to by PC
- Increment PC
  - Unless told otherwise
- Instruction loaded into Instruction Register (IR)

FETCH macro state

- Need to do
  - We need to send PC to the memory
  - Read the memory contents
  - Bring the memory contents read into the IR
  - Increment the PC
- Microstates to accomplish
  - **ifetch1**
    - **PC → MAR**
  - **ifetch2**
    - **MEM[MAR] → IR**
  - **ifetch3**
    - **PC → A**
  - **ifetch4**
    - **A+1 → PC**

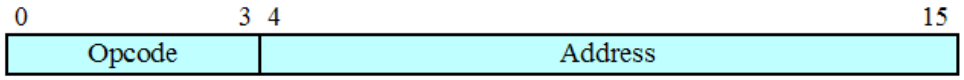
FETCH macro state (Simplifying)

- ifetch1
  - PC $\rightarrow$ MAR
  - PC $\rightarrow$ A
- ifetch2
  - MEM[MAR] $\rightarrow$ IR
- ifetch3
  - A+1 $\rightarrow$ PC

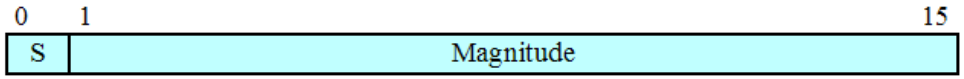
Execute Cycle

- Processor interprets instruction (decode) and performs required actions
- Actions:
  - Processor-memory
    - data transfer between CPU and main memory
  - Processor-I/O
    - Data transfer between CPU and I/O module
  - Data processing
    - Some arithmetic or logical operation on data
  - Control
    - Alteration of sequence of operations
    - e.g. jump
  - Combination of above

Varsayımsal Bir Makina (Hypothetical Machine)



(a) Instruction format



(b) Integer format

Program Counter (PC) = Address of instruction
Instruction Register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory

(d) Partial list of opcodes

Figure 1.3 Characteristics of a Hypothetical Machine

Program Çalışma Örneği

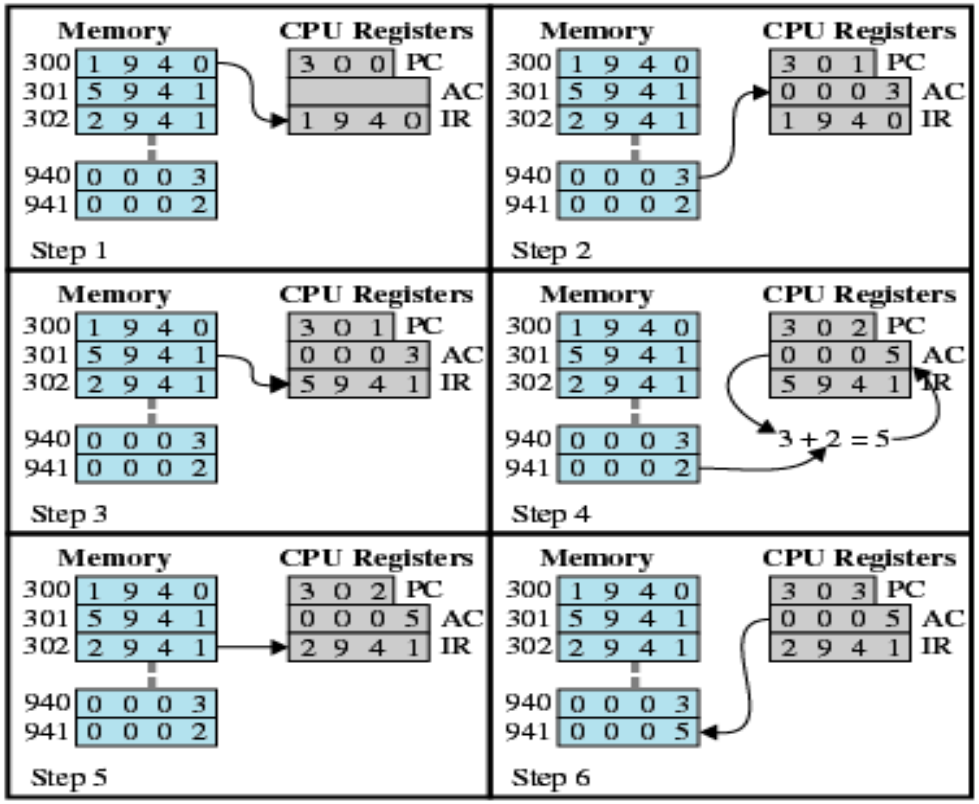
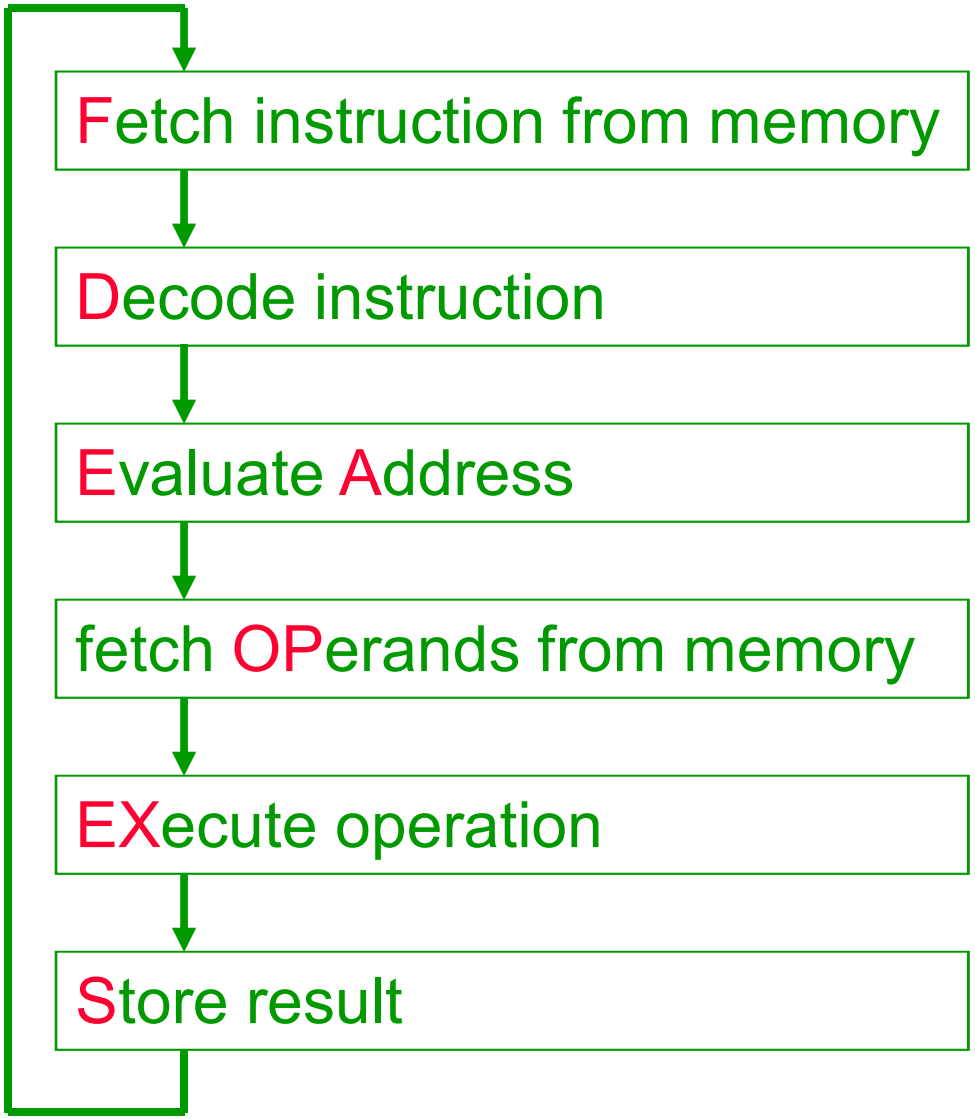


Figure 1.4 Example of Program Execution
(contents of memory and registers in hexadecimal)

Instruction Processing(Detailed)



Instruction

- The instruction is the fundamental unit of work.
- Specifies two things:
  - *opcode*: operation to be performed
  - *operands*: data/locations to be used for operation
- An instruction is encoded as a sequence of bits.
(*Just like data!*)
  - **Often, but not always, instructions have a fixed length, such as 16 or 32 bits.**
  - **Control unit interprets instruction: generates sequence of control signals to carry out operation.**
  - **Operation is either executed completely, or not at all.**

- A computer's instructions and their formats is known as its ***Instruction Set Architecture (ISA)***.

Instruction Processing: FETCH

•Load next instruction (at address stored in PC) from memory
into Instruction Register (IR).

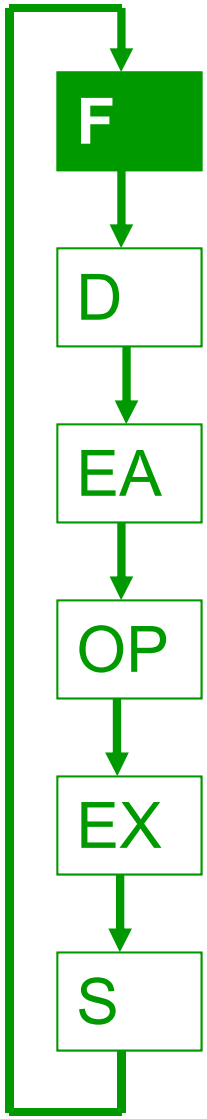
– Copy contents of PC into MAR.

– Send “read” signal to memory.

– Copy contents of MBR into IR.

•Then increment PC, so that it points to the next instruction in sequence.

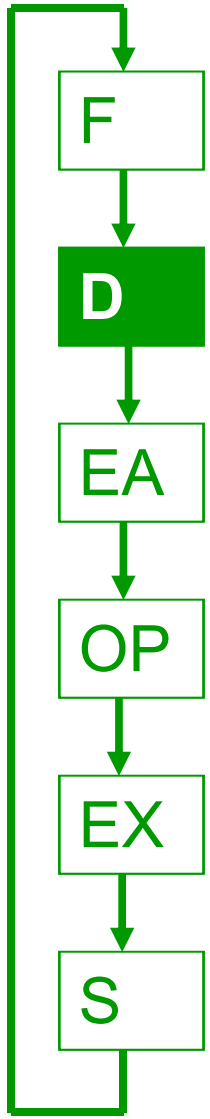
– PC becomes PC+1.



Instruction Processing: DECODE

- First identify the opcode.
  - A n-to-2^n decoder asserts a control line corresponding to the desired opcode.

- Depending on opcode, identify other operands from the remaining bits.



Instruction Processing: EVALUATE ADDRESS

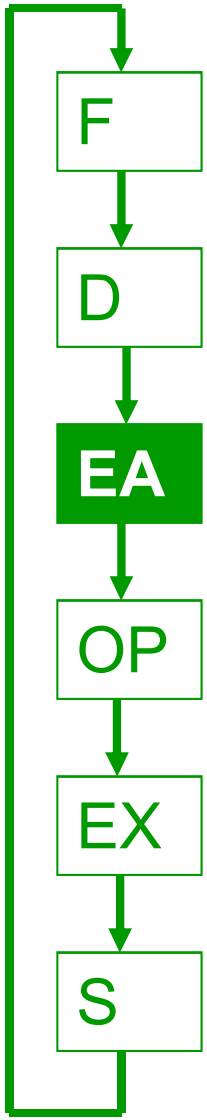
•For instructions that require memory access, compute address used for access.

•Examples:

– add offset to base register

– add offset to PC

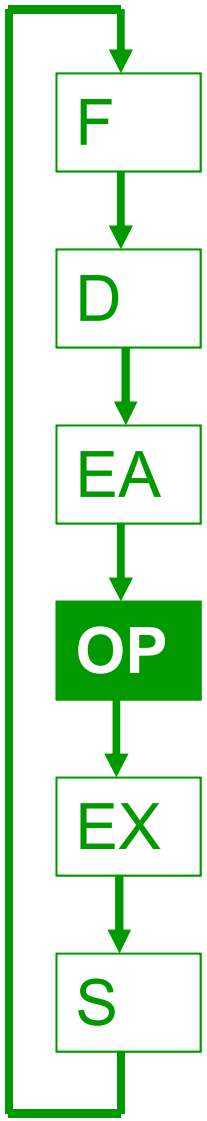
– add offset to zero



Instruction Processing: FETCH OPERANDS

- Obtain source operands needed to perform operation.

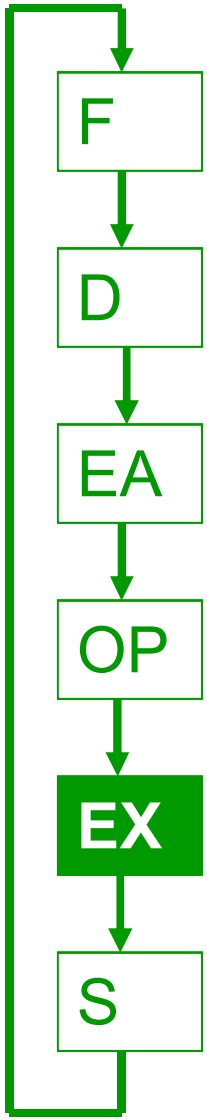
- Examples:
  - load data from memory (LDA)
  - read data from register file (ADD)



Instruction Processing: EXECUTE

- Perform the operation, using the source operands.

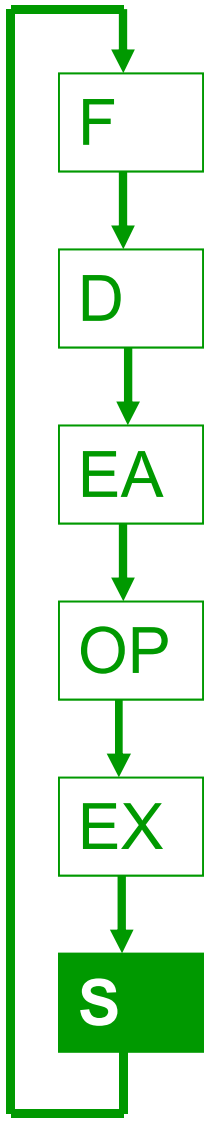
- Examples:
  - send operands to ALU and assert ADD signal
  - do nothing (e.g., for loads and stores)



Instruction Processing: STORE RESULT

- Write results to destination. (register or memory)

- Examples:
  - result of ADD is placed in destination register
  - result of memory load is placed in destination register
  - for store instruction, data is stored to memory
    - write address to MAR, data to MBR
    - assert WRITE signal to memory



Changing the Sequence of Instructions

- In the FETCH phase, we increment the Program Counter by 1.

- What if we don't want to always execute the instruction that follows this one?
  - examples: loop, if-then, function call

- Need special instructions that change the contents of the PC.
- These are called *control instructions*.
  - jumps are unconditional -- they always change the PC
  - branches are conditional -- they change the PC only if some condition is true (e.g., the result of an ADD is zero)

Instruction Processing Summary

- Instructions look just like data -- it's all interpretation.

- Four basic kinds of instructions:
  - Data processing instructions
    - Arithmetic and logic instructions (ADD, AND, …)
  - Data storage instructions
    - Memory instructions (LDA, STA)
  - Data movement instructions
    - I/O instructions (IN, OUT, …)
  - Program flow control instructions
    - Test and branch instructions (JMP, BRP, …)

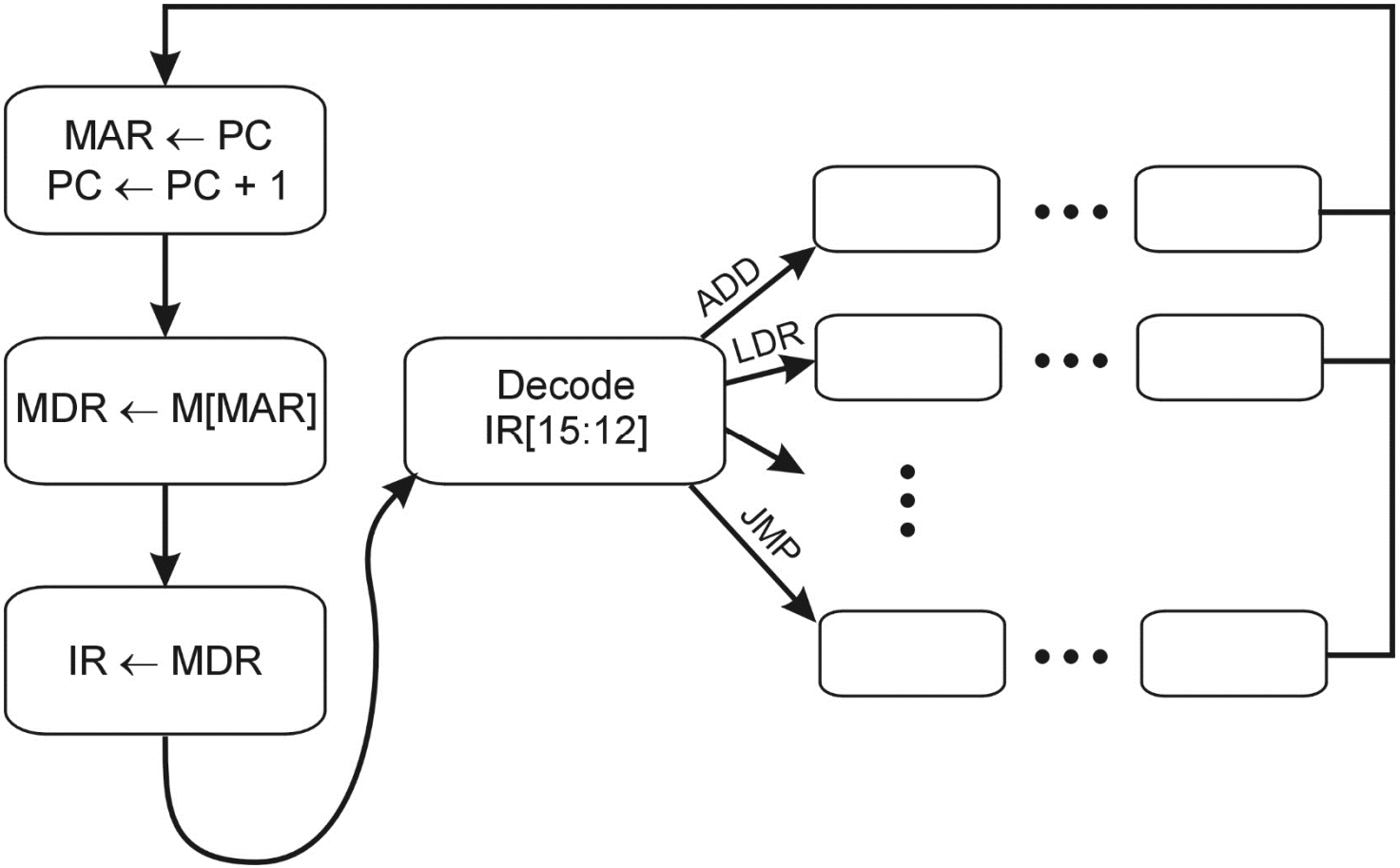
- Six basic phases of instruction processing:

  F → D → EA → OP → EX → S

  - not all phases are needed by every instruction
  - phases may take variable number of machine cycles

Control Unit State Diagram

- The control unit is a state machine.
- Here is part of a simplified state diagram:



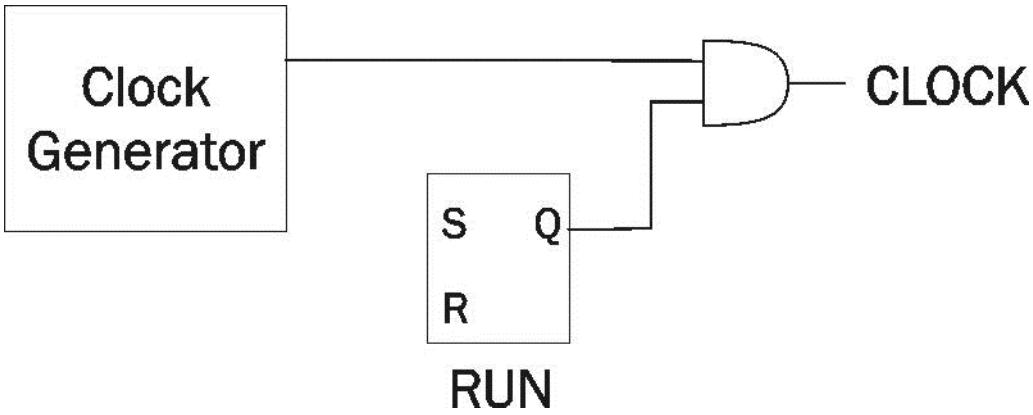
Stopping the Clock

•Control unit will repeat instruction processing sequence as long as clock is running.

- **If not processing instructions from your application, then it is processing instructions from the Operating System (OS).**
- **The OS is a special program that manages processor and other resources.**

•To stop the computer:

- **AND the clock generator signal with ZERO**
- **When control unit stops seeing the CLOCK signal, it stops processing.**



Dataflow Model of a Computer

Von Neumann model

- An instruction is fetched and executed in control flow order
  - As specified by the instruction pointer
  - Sequential unless explicit control flow instructio

Dataflow model

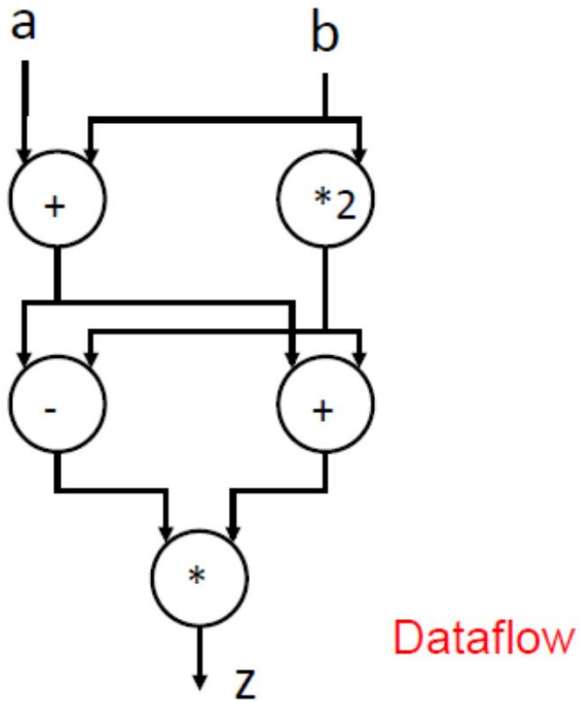
- An instruction is fetched and executed in data flow order
  - i.e., when its operands are ready
  - i.e., there is no instruction pointer
  - Instruction ordering specified by data flow dependence
    - Each instruction specifies "who" should receive the result
    - An instruction can "fire" whenever all operands are received
  - Potentially many instructions can execute at the same time
  - Inherently more parallel

von Neumann vs Dataflow

- Consider a von Neumann program (sequential) vs dataflow execution

```
v <= a + b;
w <= b * 2;
x <= v - w
y <= v + w
z <= x * y
```

Sequential



- Which model is more natural to you as a programmer?
- All major instruction set architectures today use von Neumann
  - x86, ARM, MIPS, SPARC, Alpha, POWER PC

Interrupts

- Interrupts, traps and exceptions are discontinuities in program flow
- Students asking a teacher questions in a classroom is a good analogy to the handling of discontinuities in program flow
- Mechanism by which other modules (e.g. I/O) may interrupt normal sequence of processing
- Program
  - e.g. overflow, division by zero
- Timer
  - Generated by internal processor timer
  - Used in pre-emptive multi-tasking
- I/O
  - from I/O controller
- Hardware failure
  - e.g. memory parity error

Dealing with…

- Can happen anywhere even in the middle of an instruction execution.
- Unplanned for and forced by the hardware. Hardware has to save the program counter since we are jumping to the handler.
- Address of the handler is unknown. Therefore, hardware must manufacture address.
- Since hardware saved PC, handler has to discover where to return upon completion.

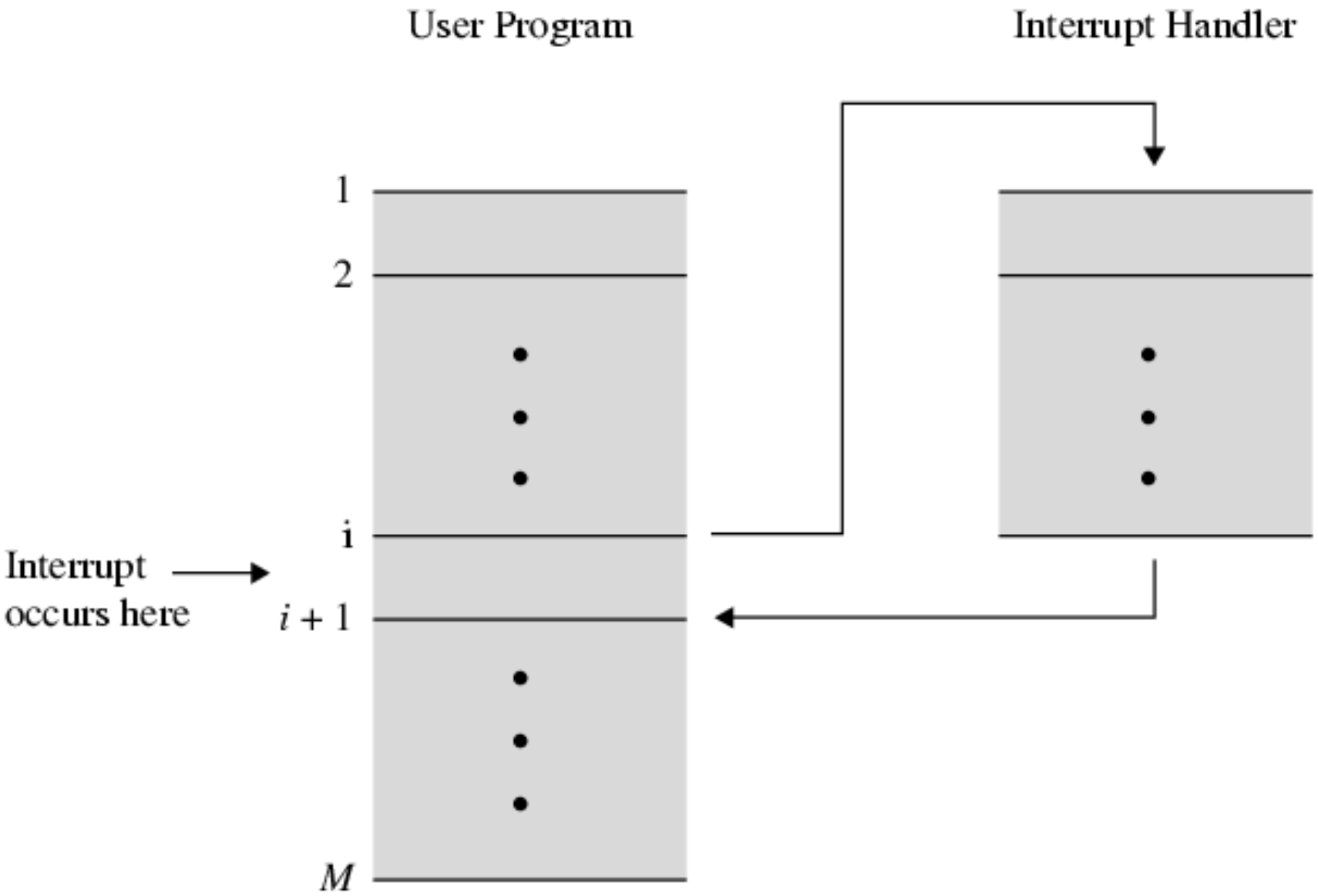
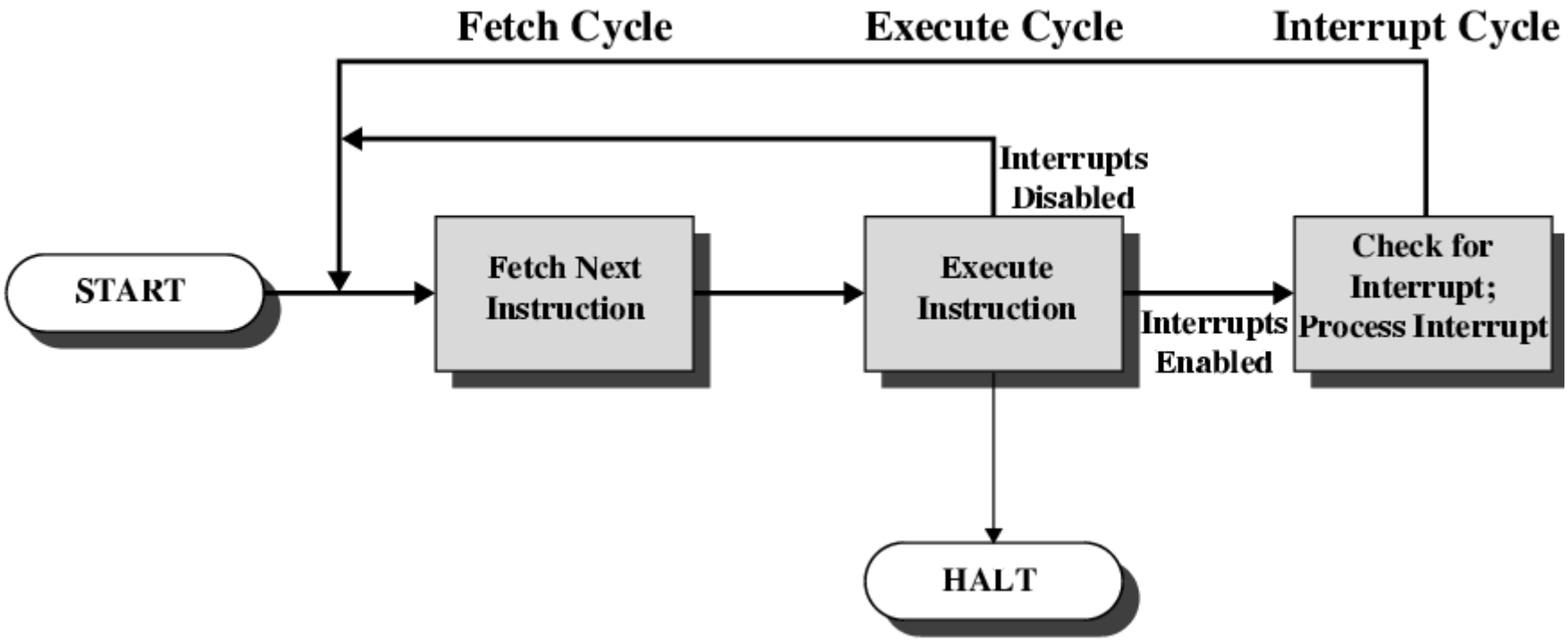
Interrupt Cycle

- Added to instruction cycle
- Processor checks for interrupt
  - Indicated by an interrupt signal
- If no interrupt, fetch next instruction
- If interrupt pending:
  - Suspend execution of current program
  - Save context
  - Set PC to start address of interrupt handler routine
  - Process interrupt
  - Restore context and continue interrupted program

A simple interrupt handler

```
Handler:
  save processor registers;
  execute device code;
  restore processor registers;
  return to original program;
```

Transfer of Control via Interrupts



Instruction Cycle with Interrupts